

Name: _____

ID: _____ Section: _____

Instructions:

- Answer all questions.
- Marks are shown in brackets, and difficulty is shown as a tag (e.g., CA-E, CA-M, CA-D).
- Understanding the question is part of the exam.
- Write the answers for Questions 1, 2(a), 2(b), 2(c), 3(c), and 3(d) in the designated answer boxes in this question paper.
- Write the answers for Questions 3(a), 3(b), 3(e), 4(a), 4(b), 4(c), and 4(d) in the answer script.
- For conceptual questions, write only the precise and relevant answer in short. Do not write detailed or descriptive explanations.

1. CO1 Process

[2]

Consider an OS running a process B, which incurs a timer interrupt at time t1. The OS context switches to another process, which performs some operations. Later, the OS context switches back to process B at time t2.

Note that process B was not run between t1 and t2.

(i) **Select** the item that is not guaranteed to be the same at times t1 and t2.

CA-M [1]

- (i) General-purpose register
- (ii) Stack pointer
- (iii) Program counter
- (iv) Process B's open file table
- (v) Kernel interrupt handler code

(ii) **Explain** the reason briefly.

CA-M [1]

Answer:

I	
II	

2. CO2 Process Synchronization

[8]

- a) An operating system uses a counting semaphore to manage access to N identical I/O devices. The initial value of the semaphore is set to N. During execution, the kernel occasionally schedules more than N processes to use the devices simultaneously, even though the total number of devices remains fixed. CA-E [2]

- (i) **Identify** which synchronization operation is being used incorrectly in this scenario? [1]
(ii) **Explain** why this problem occurs. [1]

Answer:

I	
II	

- b) A shared request queue of size N is used by multiple producer and consumer threads. The synchronization mechanism is shown below: CA-M [3]

<pre> 1 mutex = 1 2 empty = N 3 full = 0 </pre>	<pre> 1 Producer: 2 while(true){ 3 wait(mutex) 4 wait(empty) 5 6 buffer[in] = item 7 in = (in + 1) % N 8 9 signal(mutex) 10 signal(full) 11 } </pre>	<pre> 1 Consumer: 2 while(true){ 3 wait(mutex) 4 wait(full) 5 6 item = buffer[out] 7 out = (out + 1) % N 8 9 signal(mutex) 10 signal(empty) 11 } </pre>
---	---	--

- (i) **Identify** the synchronization issues in the above code and **explain** what problem may occur during execution. [2]
(ii) **Modify** the such parts of the code only which will ensure the producer-consumer synchronization works correctly. [1]

Answer:

I	
II	

- c) A distributed memory manager uses multiple threads to allocate free memory pages from a shared page table. During execution, multiple processes occasionally receive the same memory page, while some pages remain permanently marked as allocated even after release. **CA-D [3]**

```

1 page_sem = N
2 page_table[N] = {FREE}
3
4 allocate_page():
5     wait(page_sem)
6
7     for(i=0;i<N;i++){
8         if(page_table[i] == FREE){
9             page_table[i] = USED
10            page_id = i
11            break
12        }
13    }
14
15    return page_id
16
17 release_page(id):
18     page_table[id] = FREE
19
20     signal(page_sem)

```

- (i) **Identify** the synchronization issues that may occur in this implementation and **explain** why the semaphore alone cannot guarantee correct page allocation. **[2]**
- (ii) **Suggest** a proper synchronization fix for the system. **[1]**

Answer:

I	
II	

3. CO2 File Systems

[15]

- a) You are designing a miniature file system for a specific task: a bank transaction mediator. Each file contains transaction details with fixed fields of data. You settled on the block size of 256 Bytes and the INODE size of 32 Bytes. Your file storage capacity requirements are:
- It must have a maximum data storage capacity of 1 MB.
 - A maximum of 65 unique Files can be stored.
 - Follows metadata journaling, where each journal is 16 Bytes in size. You have enforced a few rules: each transaction will be processed and updated accordingly, and only after that, the next transaction can be processed; The system will be implemented as write-ahead logging, meaning the metadata will be journaled first, and after that, the actual file structure will be modified.

CA-E [4]

File System Re-gions:	SuperBlock	Journal	INODE Bitmap	Data Block Bitmap	INODE	DATA RE-GION
Block count	1 block	1 block	1 block	??	??	??

Now **write** the answers of the following questions:

- (i) How many data blocks do you need to fulfill this requirement? **[1]**
- (ii) What will be the size of the data block bitmap? **[1]**
- (iii) What will be the total blocks needed for INODE? **[1]**
- (iv) What is the amount of internal fragmentation for the INODE Bitmap? **[1]**

- b) An operating system uses a UNIX Inode file allocation method. The system is configured with a 2 KB block size and 2-byte pointers. The inode data structure contains 12 direct pointers, 1 single-indirect pointer, 1 double-indirect pointer, and 1 triple-indirect pointer. Can the file system store a file of 3.8 TB? **Justify** your answer with proper mathematical validation. **CA-M [3]**
- c) A Linux server uses a non-journaling file system that relies on fsck after crashes. While appending data to db.activity.log, the file system performs:
- Update the Data Bitmap to mark a new block allocated.
 - Write the new log data into the Data Block.
 - Update the file's Inode with the new block pointer and file size.

During heavy I/O, the server crashes after steps 1 and 2 are complete, but before the inode update is written. fsck is disabled after reboot for analysis. **CA-D [3]**

- (i) What inconsistency now exists between the inode, bitmap, and data block? **[1]**
- (ii) What will the administrator see when running cat db.activity.log? **[1]**
- (iii) What happens to disk storage over time if this keeps occurring? **[1]**

Answer:

I	
II	
III	

- d) Consider a situation where a folder spans across multiple data blocks. How does the file system know when to reclaim a particular data block belonging to the folder? **CA-D [2]**

Answer:

- e) A file system has an inode size = 128 Bytes and a block size = 1 KB. The first 5 blocks contain the superblock, journal, group descriptor, data bitmap, and inode bitmap. **CA-E [3]**
- (i) **Calculate** the address of the inode number 238. **[1]**
- (ii) The disk is sector addressable, and the size of each sector is 2 KB. **Calculate** the sector address of the inode block. **[2]**

4. CO3 Memory Management **[10]**

- a) An operating system has a logical address size of 40 bits and a page size of 16 KB. The OS can support up to 32 GB of RAM. Each row of the page table uses 3 additional bits. If it uses single-level paging, **calculate** the size of the page table. **CA-E [2]**
- b) An operating system uses a 40-bit logical address. The page size is 2 KB. Each page table entry is 2 Bytes. **Design** the hierarchical paging system required for this scenario, **showing** the logical address distribution at each level. **CA-M [2]**

- c) An operating system uses a five-level paging system, with 4 outer page tables and 1 inner page table. The system uses a TLB, where the hit ratio is 0.90. If the TLB access time = 20 ns and RAM access time = 150 ns, **calculate** the effective memory access time. **CA-D [3]**
- d) Consider a computer with a main memory that has 3 frames and page reference string of **1-6** pages: **[1 3 1 2 4 6 2 4 6 1]**. The page reference string represents the order in which the pages are accessed by a program. **CA-M [3]**
- (i) **Apply** LRU algorithm to simulate the page replacement that occurs when the main memory can hold at most 3 pages at a time. **[2.5]**
 - (ii) **Calculate** the hit ratio. **[.5]**